

PARKSIDE AI: SISTEMA INTELIGENTE DE GESTIÓN DE ESTACIONAMIENTO CON RECONOCIMIENTO DE PLACAS Y PAGO AUTOMÁTICO

Resumen -- Este proyecto presenta ParkSide AI, un sistema inteligente de gestión de estacionamientos basado en una arquitectura modular y escalable. La innovación principal radica en la automatización del acceso vehicular mediante reconocimiento inteligente y la gestión de pagos a través de métodos pre-registrados, optimizando la eficiencia y la experiencia del usuario. El sistema busca conceptualizar una solución moderna y automatizada para la gestión de estacionamientos, demostrando la aplicación de principios de arquitectura de software para un caso de uso práctico.

I. INTRODUCCIÓN

La gestión de estacionamientos, en entornos urbanos y comerciales modernos, a menudo se caracteriza por procesos manuales, congestión vehicular y experiencias de usuario subóptimas. La necesidad de soluciones más eficientes, rápidas y convenientes se ha vuelto primordial para optimizar el flujo de vehículos, reducir los tiempos de espera y mejorar la satisfacción tanto de los usuarios como de los operadores de estacionamientos.

En este contexto, se presenta ParkSide AI, un sistema inteligente de gestión de estacionamientos concebido bajo una arquitectura de software robusta y escalable. Este proyecto busca revolucionar la experiencia de estacionamiento mediante la integración de capacidades de reconocimiento inteligente para la automatización del acceso vehicular y la gestión de pagos sin fricciones. La premisa fundamental de ParkSide AI es la creación de un ecosistema de estacionamiento donde la tecnología actúe de manera proactiva para facilitar cada etapa del proceso, desde la entrada hasta la salida.

La conceptualización de ParkSide AI se centra en la aplicación de principios sólidos de arquitectura de software para construir un sistema modular, mantenible y adaptable a las diversas necesidades de los entornos de estacionamiento. Al priorizar la automatización y la inteligencia en la toma de decisiones, ParkSide AI aspira a minimizar la intervención manual, reducir los errores humanos y proporcionar datos valiosos para la optimización operativa.

El presente informe técnico tiene como objetivo ofrecer una visión general del sistema ParkSide AI, desde su concepción y

los desafíos que busca abordar, hasta la arquitectura general propuesta y las funcionalidades clave que lo definen. A través de esta exploración, se busca establecer una comprensión clara del potencial de ParkSide AI para transformar la gestión de estacionamientos, sentando las bases para un análisis más detallado de sus componentes y su implementación.

II. DEFINICIÓN DEL PROBLEMA Y EQUIPO

A. Definición del problema

La gestión contemporánea de estacionamientos, tanto en entornos urbanos de alta densidad como en instalaciones comerciales y privadas, enfrenta una serie de ineficiencias y desafíos que impactan negativamente tanto a los usuarios como a los operadores. Estos problemas se manifiestan en múltiples facetas del proceso de estacionamiento, desde la entrada y la búsqueda de plazas hasta el pago y la salida, generando congestión, frustración y pérdidas económicas.

Ineficiencias en el Acceso y Salida: Los métodos tradicionales de acceso y salida, basados en tickets físicos, tarjetas de proximidad o incluso la intervención manual de personal, suelen ser lentos y propensos a errores. La necesidad de detener el vehículo, interactuar con una máquina o un empleado, y esperar la validación o el pago, contribuye significativamente a la congestión vehicular en las entradas y salidas, especialmente en horas pico. La pérdida o el daño de los tickets físicos también generan inconvenientes y retrasos adicionales.

Proceso de Pago Inconveniente y Lento: Los sistemas de pago convencionales a menudo requieren que los usuarios hagan fila en cajas designadas, busquen efectivo o utilicen tarjetas, lo que consume tiempo valioso y puede generar frustración. La falta de opciones de pago diversificadas y la ausencia de procesos automatizados contribuyen a la ineficiencia y a una experiencia de usuario insatisfactoria.

Gestión Operativa Ineficiente: Para los operadores de estacionamientos, la gestión manual de los ingresos, la supervisión de la ocupación, el control de los tiempos de estancia y la conciliación de pagos pueden ser procesos laboriosos y costosos. La falta de datos precisos y en tiempo real

dificulta la toma de decisiones estratégicas para la optimización de precios, la gestión de la capacidad y la mejora de la eficiencia operativa.

Seguridad y Control de Acceso Limitados: Los sistemas de acceso tradicionales pueden presentar vulnerabilidades en términos de seguridad y control. La falsificación de tickets o el acceso no autorizado pueden generar pérdidas económicas para los operadores y comprometer la seguridad de las instalaciones.

1) Objetivos Establecidos con ParkSide AI:

Para abordar estas ineficiencias y desafíos, el sistema ParkSide AI se ha concebido con los siguientes objetivos primordiales:

1. **Automatizar el Acceso y Salida Vehicular:** Implementar un sistema que permita la entrada y salida de vehículos de forma automática y sin intervención manual, utilizando tecnologías de reconocimiento inteligente.
2. **Optimizar el Proceso de Pago:** Facilitar un sistema de pago ágil y sin fricciones, mediante la gestión de métodos de pago pre-registrados y la automatización del débito o descuento al momento de la salida.
3. **Mejorar la Experiencia del Usuario:** Proporcionar una experiencia de estacionamiento más rápida, cómoda y eficiente, reduciendo los tiempos de espera y la frustración.
4. **Incrementar la Eficiencia Operativa:** Ofrecer a los operadores de estacionamientos datos precisos y en tiempo real sobre la ocupación, los ingresos y el flujo vehicular, facilitando la toma de decisiones informadas y la optimización de la gestión.
5. **Fortalecer la Seguridad y el Control de Acceso:** Implementar un sistema de identificación vehicular robusto y seguro, minimizando el riesgo de accesos no autorizados y fraudes.
6. **Contribuir a la Sostenibilidad:** Reducir la congestión vehicular en las inmediaciones de los estacionamientos y dentro de ellos, disminuyendo las emisiones contaminantes y el consumo de combustible innecesario.

2) ParkSide AI como Solución al Problema:

ParkSide AI se presenta como una solución integral y efectiva a los problemas definidos anteriormente mediante la aplicación de tecnologías avanzadas y principios de arquitectura de software sólidos:

- **Automatización Inteligente del Acceso:** La integración de la detección de placas basada en Inteligencia Artificial elimina la necesidad de tickets o tarjetas, permitiendo un flujo vehicular más rápido y eficiente en las entradas y salidas.

- **Pagos Automatizados y sin Contacto:** La gestión de métodos de pago pre-registrados y el débito automático simplifican el proceso de pago, eliminando las filas y los puntos de contacto físico, mejorando la comodidad y la seguridad.
- **Mayor Seguridad y Control:** El reconocimiento de placas ofrece un método de identificación vehicular más robusto y trazable, fortaleciendo la seguridad y el control de acceso a las instalaciones.

En esencia, ParkSide AI busca transformar la gestión de estacionamientos de un proceso manual y a menudo problemático a una experiencia fluida, automatizada e inteligente, beneficiando tanto a los usuarios finales como a los operadores de las instalaciones. La aplicación de una arquitectura de software bien definida y la integración de tecnologías de vanguardia son los pilares fundamentales que sustentan la capacidad de ParkSide AI para abordar eficazmente los desafíos existentes en la gestión moderna de estacionamientos.

B. Roles del equipo

1. Andrés Hurtado - director general del Proyecto y Especialista en Inteligencia Artificial

- **Rol Principal:** director General del Proyecto.
- **Responsabilidades:**
 - **Dirección General:** Liderazgo y visión general del proyecto, asegurando la alineación con los objetivos establecidos y la correcta gestión de los recursos.
 - **Scrum Master:** Facilitación de las ceremonias Scrum (sprints, reuniones diarias, retrospectivas, etc.), gestión del backlog del producto, eliminación de impedimentos para el equipo y aseguramiento del cumplimiento de los tiempos y las tareas planificadas.
 - **Inteligencia Artificial:** Investigación, diseño, desarrollo e implementación de los modelos de Inteligencia Artificial necesarios para la detección y reconocimiento de placas vehiculares. Esto incluye la selección de algoritmos, entrenamiento de modelos, optimización del rendimiento y la integración con el resto del sistema.

2. Mateo Fonseca - Arquitecto de Datos y Desarrollador Backend con Integración IA

- **Rol Principal:** Arquitecto de Datos y Desarrollador Backend.

- **Responsabilidades:**

- **Arquitectura de Datos:** Diseño y modelado de la arquitectura de datos del sistema, incluyendo la definición de las estructuras de almacenamiento, el flujo de datos y las políticas de gestión de la información.
- **Diseño de la Base de Datos:** Creación y gestión del esquema de la base de datos, asegurando su eficiencia, escalabilidad e integridad para almacenar la información de usuarios, vehículos, métodos de pago, registros de estacionamiento, etc.
- **Desarrollo Backend:** Desarrollo de la lógica del lado del servidor utilizando las tecnologías definidas, incluyendo la creación de APIs, la implementación de la lógica de negocio y la gestión de la comunicación con la base de datos.
- **Integración con IA:** Colaboración estrecha con Andrés Hurtado en la integración de los modelos de Inteligencia Artificial desarrollados en el backend del sistema, asegurando una comunicación eficiente y el correcto flujo de datos para el reconocimiento de placas.

3. Thomas Ariza - Desarrollador Backend y Responsable de Pruebas

- **Rol Principal:** Desarrollador Backend y Responsable de Pruebas.
- **Responsabilidades:**
 - **Desarrollo Backend:** Desarrollo de la lógica del lado del servidor, contribuyendo a la creación de APIs y la implementación de la lógica de negocio.
 - **Pruebas:** Diseño, desarrollo y ejecución de pruebas para los diferentes servicios y componentes del backend, asegurando la calidad, la fiabilidad y el correcto funcionamiento individual de cada módulo.
 - **Testeo de Servicios:** Colaboración con Mateo Fonseca en la realización de pruebas de integración entre los diferentes servicios del backend.
 - **Integración con IA:** Colaboración con Andrés Hurtado y Mateo Fonseca en la verificación e integración de los componentes de Inteligencia Artificial en el backend.

4. Juan Valdés - Desarrollador Frontend y Especialista en UX/UI

- **Rol Principal:** Desarrollador Frontend y Especialista en Experiencia de Usuario (UX) e Interfaz de Usuario (UI).
- **Responsabilidades:**
 - **Desarrollo Frontend:** Desarrollo de la interfaz de usuario para las diferentes plataformas (aplicación móvil para usuarios, panel de administración para operadores) utilizando las tecnologías definidas.
 - **Diseño UX/UI:** Responsabilidad del diseño de la experiencia de usuario, incluyendo la arquitectura de la información, la usabilidad, la accesibilidad y el diseño visual de la interfaz de usuario, asegurando una experiencia intuitiva y atractiva para los usuarios.
 - **Prototipado:** Creación de prototipos y mockups para validar las ideas de diseño y la funcionalidad antes de la implementación final.

5. Manuel Castro - Desarrollador FullStack (Arquitectura, Integración y Seguridad)

- **Rol Principal:** Desarrollador FullStack (Arquitectura, Integración y Seguridad).
- **Responsabilidades:**
 - **Arquitectura Frontend:** Definición y diseño de la arquitectura del frontend, asegurando su escalabilidad, modularidad y rendimiento.
 - **Integración Backend:** Responsabilidad de la integración del frontend con el backend, asegurando una comunicación eficiente y el correcto consumo de las APIs desarrolladas.
 - **Creación de Modelos Frontend:** Definición y gestión de los modelos de datos y el manejo del estado en el frontend.
 - **Gestión de Pruebas:** Diseño y ejecución de pruebas de seguridad a nivel del frontend para identificar y mitigar posibles vulnerabilidades, colaborando con el equipo de backend en la seguridad integral del sistema.

III. ARQUITECTURA

A. Arquitecturas implementadas

La arquitectura del sistema ParkSide AI se ha diseñado considerando la naturaleza distribuida de sus funcionalidades, la necesidad de escalabilidad, la mantenibilidad a largo plazo y la optimización del rendimiento en cada una de sus capas. A continuación, se detalla la arquitectura adoptada para el Frontend, el Backend y el componente de Inteligencia Artificial, junto con la justificación de estas decisiones.

1) Arquitectura del Frontend: Basada en Componentes Individuales con Modularización (Layouts y Views)

- **Descripción:** El Frontend de ParkSide AI se estructura mediante una arquitectura basada en componentes individuales y altamente reutilizables. Estos componentes se organizan jerárquicamente en módulos funcionales, layouts que definen la estructura visual de las páginas, y views que representan las diferentes pantallas o secciones de la aplicación.
- **Justificación de la Elección:**
 - **Reusabilidad y Mantenibilidad:** La arquitectura basada en componentes promueve la creación de elementos de interfaz reutilizables en diferentes partes de la aplicación, lo que reduce la duplicación de código y facilita el mantenimiento a largo plazo. Las actualizaciones o modificaciones en un componente específico tienen un impacto localizado, simplificando el proceso de desarrollo y corrección de errores.
 - **Modularidad y Escalabilidad:** La organización en módulos, layouts y views facilita la división del trabajo entre los desarrolladores y permite escalar la aplicación de manera más sencilla. Nuevas funcionalidades o secciones pueden implementarse como módulos independientes sin afectar la estabilidad del resto del sistema.
 - **Experiencia de Usuario Consistente:** El uso de layouts predefinidos asegura una estructura visual coherente en toda la aplicación, contribuyendo a una mejor experiencia de usuario. Los componentes individuales, al ser reutilizables, también ayudan a mantener un estilo y un comportamiento consistentes.
 - **Facilita el Desarrollo y las Pruebas:** La naturaleza aislada de los componentes individuales facilita el desarrollo paralelo y la realización de pruebas unitarias específicas para cada componente, asegurando una mayor calidad del código.

2) Arquitectura del Backend: Orientada a Servicios (SOA)

- **Descripción:** El Backend de ParkSide AI se implementa siguiendo una arquitectura orientada a servicios (SOA). Esto implica la descomposición de la lógica de negocio en una colección de servicios independientes y autónomos, cada uno responsable de una funcionalidad específica (gestión de usuarios, gestión de pagos, registro de vehículos, etc.). Estos servicios se comunican entre sí a través de endpoints bien definidos, y todos pueden acceder a la misma base de datos compartida.
- **Justificación de la Elección:**
 - **Escalabilidad Independiente:** La SOA permite escalar cada servicio de forma independiente según sus necesidades de carga. Por ejemplo, el servicio de gestión de pagos podría requerir más recursos en ciertos momentos que el servicio de gestión de usuarios.
 - **Mantenibilidad y Reusabilidad de Servicios:** Cada servicio es un componente autónomo que puede ser desarrollado, probado, desplegado y mantenido de forma independiente. Esto facilita la gestión del código y promueve la reusabilidad de los servicios en diferentes partes de la aplicación o incluso en otros sistemas en el futuro.
 - **Flexibilidad Tecnológica:** La SOA permite utilizar diferentes tecnologías o lenguajes de programación para implementar cada servicio, aunque en este caso se utilice principalmente FastAPI. Esto ofrece flexibilidad y la posibilidad de elegir la mejor herramienta para cada tarea específica.
 - **Robustez y Tolerancia a Fallos:** Si un servicio falla, el resto del sistema puede seguir funcionando (aunque la funcionalidad dependiente de ese servicio no esté disponible). Esto mejora la robustez general del sistema.
 - **Facilita la Integración:** Los servicios con endpoints bien definidos facilitan la integración con otros sistemas o aplicaciones en el futuro.

3) Arquitectura del Componente de Inteligencia Artificial: Orientada a Eventos

- **Descripción:** El componente de Inteligencia Artificial para la detección de placas vehiculares se basa en una

arquitectura orientada a eventos. Esto significa que el sistema reacciona y procesa la información en función de la llegada de eventos específicos, en este caso, la detección de la presencia de un vehículo por la cámara. Cuando la cámara captura una imagen y el software de IA identifica una placa, se genera un evento que desencadena el procesamiento posterior (registro de entrada, verificación, etc.).

- **Justificación de la Elección:**

- **Naturaleza Reactiva y en Tiempo Real:** La detección de vehículos es un evento en tiempo real que requiere una respuesta inmediata. La arquitectura orientada a eventos es ideal para este tipo de procesamiento asíncrono y reactivo.
- **Desacoplamiento:** El componente de IA puede operar de forma independiente y emitir eventos sin necesidad de conocer los detalles de cómo esos eventos serán procesados por otros componentes del sistema. Esto facilita la escalabilidad y la mantenibilidad.
- **Flexibilidad y Extensibilidad:** Nuevas funcionalidades o procesos pueden suscribirse a los eventos generados por el componente de IA sin necesidad de modificar el componente de IA en sí. Por ejemplo, se podría añadir un sistema de notificación o un registro de auditoría que reaccione al evento de detección de placa.
- **Rendimiento:** El procesamiento basado en eventos puede ser más eficiente en términos de recursos, ya que los componentes solo se activan cuando ocurre un evento relevante.

En conjunto, la combinación de una arquitectura basada en componentes para el Frontend, una arquitectura orientada a servicios para el Backend y una arquitectura orientada a eventos para el componente de Inteligencia Artificial permite construir un sistema ParkSide AI robusto, escalable, mantenible y eficiente, optimizado para las necesidades específicas de cada una de sus capas funcionales. La elección de estas arquitecturas se basa en los principios de diseño de software que buscan la separación de responsabilidades, la reusabilidad, la escalabilidad y la adaptabilidad a los requerimientos del sistema y a las futuras evoluciones.

IV. BACKEND

A. APIs implementadas

1. Autenticación y autorización (Login)

Ruta: /login/access-token

Método: POST

Descripción:

Permite a los usuarios autenticarse utilizando sus credenciales (email y contraseña). Si las credenciales son correctas, el servidor devuelve un JWT (token de acceso) que el cliente utilizará para realizar peticiones a otras rutas protegidas.

Parámetros de entrada:

email: Correo electrónico del usuario.

password: Contraseña en texto plano (se compara con el hash de la contraseña almacenada en la base de datos).

Respuesta:

Inicio de sesión exitoso

Flujo:

El sistema recibe el email y password.

Verifica que el email esté registrado en la base de datos.

Compara la contraseña proporcionada con el hash almacenado utilizando bcrypt.

Si la autenticación es exitosa, genera un Hash lip con una fecha de expiración configurada y lo devuelve.

2. Recuperación de contraseña

Ruta: /password-recovery/{email}

Método: POST

Descripción:

Permite al usuario recuperar su contraseña. El sistema genera un token de recuperación y envía un correo electrónico con un enlace para restablecer la contraseña.

- **Parámetros de entrada:**

- email: El correo electrónico del usuario que solicita la recuperación de la contraseña.

- **Respuesta:**

- message: Un mensaje confirmando que el correo de recuperación fue enviado.

Ruta: /reset-password/

Método: POST

Descripción:

Permite al usuario restablecer su contraseña usando el token recibido en el correo electrónico de recuperación.

- **Parámetros de entrada:**

- token: El token de recuperación proporcionado en el correo electrónico.
- new_password: La nueva contraseña del usuario.

- **Respuesta:**

- message: Un mensaje confirmando que la contraseña ha sido restablecida correctamente.

3. Gestión de tarjetas

Ruta: /cards/register

Método: POST

Descripción:

Permite registrar una nueva tarjeta para un usuario.

- **Parámetros de entrada:**
 - card_number: Número de la tarjeta (debe estar encriptado).
 - cvc_code: Código CVC de la tarjeta (debe estar encriptado).
 - full_name_customer: Nombre completo del titular de la tarjeta.
 - expiration_date: Fecha de vencimiento de la tarjeta.
 - card_type: Tipo de tarjeta (Visa, MasterCard, etc.).
- **Respuesta:**
 - message: Confirmación de que la tarjeta ha sido registrada exitosamente.

4. Gestión de vehículos

Ruta: /vehicles/register

Método: POST

Descripción:

Permite registrar un nuevo vehículo para un cliente.

- **Parámetros de entrada:**
 - plate: Placa del vehículo.
 - type: Tipo de vehículo (carro, moto, etc.).
- **Respuesta:**
 - message: Confirmación de que el vehículo ha sido registrado exitosamente.

5. Gestión de pagos

Ruta: /payments/

Método: POST

Descripción:

Permite registrar un nuevo pago realizado por el usuario.

- **Parámetros de entrada:**
 - status: Estado del pago (pendiente, aprobado, rechazado).
 - status_detail: Detalles adicionales sobre el estado del pago.
 - payment_method: Método de pago utilizado (tarjeta, transferencia, etc.).
 - payment_type: Tipo de pago (completo, parcial).
 - transaction_amount: Monto de la transacción.
 - installments: Número de cuotas (si aplica).
 - parking_registration_id: ID del registro de parqueo asociado con el pago.
 - employee_id: ID del empleado que gestionó el pago (si aplica).
- **Respuesta:**
 - message: Confirmación de que el pago ha sido registrado exitosamente.

6. Gestión de parqueos

Ruta: /parking/

Método: POST

Descripción:

Permite registrar un nuevo parqueadero.

- **Parámetros de entrada:**
 - name: Nombre del parqueadero.
 - places: Número de espacios disponibles.
 - address: Dirección del parqueadero.
 - enterprise: Nombre de la empresa operadora del parqueadero.
- **Respuesta:**
 - message: Confirmación de que el parqueadero ha sido registrado exitosamente.

7. Gestión de registros de parqueo

Ruta: /parking-registration/

Método: POST

Descripción:

Permite registrar el ingreso y egreso de un vehículo en un parqueadero.

- **Parámetros de entrada:**
 - entry_datetime: Fecha y hora de ingreso del vehículo al parqueadero.
 - exit_datetime: Fecha y hora de egreso del vehículo (si aplica).
 - plate: Placa del vehículo.
- **Respuesta:**
 - message: Confirmación de que el registro de parqueo ha sido exitoso.

8. Historial de tarifas

Ruta: /historical-rate/

Método: POST

Descripción:

Permite registrar el historial de tarifas de un parqueadero.

- **Parámetros de entrada:**
 - car_rate: Tarifa por estacionar un carro.
 - motorbike_rate: Tarifa por estacionar una moto.
 - start_date: Fecha de inicio de la tarifa.
 - end_date: Fecha de fin de la tarifa (si aplica).
 - parking_id: ID del parqueadero al que se aplica la tarifa.
- **Respuesta:**
 - message: Confirmación de que el historial de tarifas ha sido registrado exitosamente.

B. Flujo de datos

El sistema ParkSide AI organiza el flujo de datos en módulos independientes pero interconectados que permiten la trazabilidad completa de un vehículo desde su registro hasta el flujo de entrada/salida del parqueadero y su respectivo pago. Este flujo se divide principalmente en tres dominios: gestión de tarjetas, gestión de vehículos y gestión del parqueo, estructurados bajo una arquitectura orientada a servicios (SOA).

1. Registro de Vehículo

El módulo vehicle.py (API) recibe las solicitudes del cliente con los datos del vehículo, que son validados y procesados por el CRUD vehicleCrud.py. Esta lógica se conecta al modelo vehicle.py (models), el cual define la estructura del vehículo y su relación con el usuario. Una vez verificados, los datos se almacenan en la base de datos.

2. Gestión de Tarjetas

Similarmente, el módulo cardManage.py actúa como punto de entrada para las solicitudes de registro de tarjetas. Este invoca a cardCrud.py, que se encarga de encriptar los datos sensibles (número, CVC) y almacenarlos utilizando el modelo card.py. Este modelo establece la relación entre el cliente y sus tarjetas activas, asegurando un uso seguro en operaciones de pago.

3. Registro de Entrada y Salida de Vehículos

Cuando un vehículo es detectado en el ingreso al parqueadero, el sistema genera un evento que es manejado por flowRegistration.py, el cual decide si se trata de una entrada o salida. Esta lógica invoca a flowRegistrationCrud.py para registrar o actualizar la entrada/salida en la base de datos, usando como referencia la placa del vehículo y su información previamente registrada en vehicle.py.

4. Gestión del Parqueo

El módulo parkingRegistration.py actúa como interfaz entre la entrada/salida y el historial de ocupación. Este módulo registra el tiempo de entrada y salida de cada vehículo, validando si el mismo se encuentra registrado y si cuenta con un método de pago activo. Los datos son consolidados en las tablas mediante el uso de los modelos interrelacionados (vehicle, parking_registration, payment).

5. Secuencia General del Flujo

- El usuario registra su vehículo → se guarda en vehicleCrud.py y vehicle.py (models)
- Registra una tarjeta de pago → procesado por cardCrud.py y guardado en card.py (models)
- Al ingresar al parqueadero → flowRegistration.py recibe la placa y llama a flowRegistrationCrud.py
- Se consulta si el vehículo existe → si existe, se registra la entrada en parkingRegistration.py

- Al salir → se repite el proceso y se calcula el total del tiempo y se asocia con un pago.

Este sistema modular garantiza la trazabilidad, integridad de los datos y seguridad, permitiendo que cada transacción de ingreso, salida y pago quede registrada y validada en tiempo real, fortaleciendo el control operativo del parqueadero.

C. Roles y autorización

En el sistema, la autenticación y autorización están basadas en el hashing bcrypt para las contraseñas. A continuación, se detalla cómo se gestionan los roles y la autorización en el contexto de este enfoque:

Autenticación Basada en Hashing:

Las contraseñas de los usuarios se hash con bcrypt, un algoritmo de hashing seguro.

Cuando un usuario intenta iniciar sesión, el backend recibe la contraseña en texto claro, la compara con el hash almacenado en la base de datos, y si coinciden, se autentica correctamente.

Si la contraseña proporcionada por el usuario no coincide con el hash de la base de datos o si el usuario no existe, el acceso es denegado.

Roles y Autorización:

Los roles de usuario son asignados cuando se crea una cuenta. Los roles podrían incluir, pero no se limitan a:

Cliente: Tiene acceso a funcionalidades como visualizar su historial de pagos, registrar vehículos, y gestionar sus tarjetas.

Empleado: Tiene permisos para gestionar pagos, crear registros de parqueos, y otras funciones administrativas limitadas.

Administrador: Tiene acceso a todas las funcionalidades del sistema, incluyendo la gestión de parqueaderos, tarifas, empleados, y otros usuarios.

Los roles de los usuarios se almacenan en la base de datos y se verifican durante cada solicitud a rutas protegidas.

Protección de Rutas con Roles:

Cada ruta del backend tiene restricciones basadas en roles. Por ejemplo:

Las rutas para gestionar parqueaderos o gestionar tarifas son accesibles solo para administradores.

Las rutas de registro de vehículos y gestión de tarjetas están disponibles para clientes autenticados.

La autorización en el backend se gestiona mediante la verificación del rol del usuario durante el proceso de autorización.

Middleware de Autorización:

El sistema incluye un middleware de autorización que valida que el usuario tenga el rol adecuado para acceder a una ruta específica.

El middleware intercepta las solicitudes entrantes, verifica la autenticación (hashing bcrypt) y asegura que el rol del usuario tenga acceso a la ruta solicitada.

Si un usuario intenta acceder a una ruta para la cual no tiene permisos (por ejemplo, un cliente intentando acceder a rutas de administrador), el sistema devuelve un error 403 Forbidden.

Restricción de Rutas y Funcionalidades:

Las rutas más sensibles, como las de gestionar pagos, modificar tarifas o gestionar empleados, están restringidas a ciertos roles:

Clientes solo pueden realizar solicitudes que afecten su propia información (como registrar vehículos, consultar su historial de parqueo, etc.).

Empleados tienen permisos para registrar pagos y asociar parqueos a clientes.

Administradores tienen acceso total, incluyendo la gestión de todos los usuarios y configuración del sistema.

Verificación de Roles:

La verificación del rol se realiza en el backend, donde el sistema obtiene el rol del usuario autenticado y determina si tiene permisos suficientes para acceder a la ruta.

Se hace una verificación de autenticación con bcrypt (para asegurarse de que el usuario es quien dice ser), y luego se realiza una autorización para asegurar que el rol de ese usuario permita el acceso a esa ruta.

Consideraciones de Seguridad en la Autorización:

Contraseñas Seguras: Las contraseñas no se almacenan ni se transmiten en texto claro. Son hashadas utilizando bcrypt antes de ser almacenadas.

Control de Acceso: El sistema garantiza que solo los usuarios con roles específicos pueden acceder a funcionalidades críticas, como gestionar pagos, parqueos, y empleados.

Protección contra Accesos No Autorizados: En el caso de roles no válidos o usuarios no autenticados, el sistema

responde con un error adecuado (como 403 Forbidden o 401 Unauthorized).

Flujo de Autenticación y Autorización:

El usuario ingresa su correo y contraseña.

El hash de la contraseña se compara con el valor almacenado en la base de datos utilizando bcrypt.

Si la autenticación es exitosa, el sistema verifica el rol del usuario.

Según el rol, el acceso a las rutas protegidas se decide. Si el usuario tiene el rol adecuado, puede acceder a las funcionalidades permitidas por su rol.

Escalabilidad y Mantenimiento de Roles:

Los roles pueden ser fácilmente gestionados en la base de datos, y se pueden agregar más roles según la necesidad del sistema.

La implementación de roles en el sistema es flexible, permitiendo la adaptación a nuevos requisitos de seguridad o funcionalidades.

D. Validaciones de seguridad

Autenticación con Hashing de Contraseñas (Bcrypt):

- Las contraseñas no se almacenan ni se transmiten en texto plano. En su lugar, se utiliza **bcrypt** para crear un hash seguro de las contraseñas, y este hash se almacena en la base de datos.
- Durante el proceso de autenticación, cuando un usuario intenta iniciar sesión, la contraseña proporcionada se compara con el hash almacenado en la base de datos utilizando bcrypt.
- Si el hash de la contraseña proporcionada coincide con el hash almacenado, el acceso se concede; de lo contrario, el acceso es rechazado.

Autorización basada en Roles:

- Los roles de los usuarios (como **cliente**, **empleado**, o **administrador**) se verifican antes de permitir el acceso a rutas específicas.
- Solo los usuarios con los roles adecuados tienen acceso a rutas que gestionan datos sensibles, como la creación de parqueaderos, la actualización de tarifas, o la gestión de pagos.

Protección de Rutas Sensibles:

- Las rutas críticas como el acceso a datos de tarjetas de crédito o información personal se protegen para asegurar que solo los usuarios con los permisos adecuados (basados en sus roles) puedan acceder a ellas.
- Esto incluye rutas de **registro de vehículos**, **gestión de pagos** y **gestión de parqueaderos**.

Validación de Datos de Entrada:

- Todos los datos de entrada del usuario, como **números de tarjeta**, **fechas de vencimiento** y

números de placa de vehículos, son validados tanto en el frontend como en el backend para evitar inyecciones de código malicioso (XSS, SQL Injection).

- Además, los datos se sanitizan para garantizar que no contengan caracteres o formatos no esperados.

Protección contra Ataques de Fuerza Bruta:

- El backend implementa un **limitador de intentos** para evitar ataques de **fuerza bruta** en el proceso de autenticación. Tras varios intentos fallidos de inicio de sesión, el sistema bloquea temporalmente la cuenta del usuario o aplica un retraso antes de permitir nuevos intentos.

Cifrado de Datos Sensibles:

- Los datos sensibles, como los **números de tarjeta de crédito** y los **códigos CVV**, se cifran antes de ser almacenados en la base de datos, de modo que incluso si hay una fuga de datos, la información sigue siendo segura.

V. FRONTEND

Las aplicaciones de ParkSide AI se ha diseñado con una atención primordial en la experiencia del usuario final, buscando ofrecer una interacción intuitiva, eficiente y visualmente agradable. Cada elemento de la interfaz ha sido cuidadosamente considerado para facilitar la navegación, la comprensión y la realización de las tareas clave dentro del sistema de gestión de estacionamientos automatizado.

A. Diseño de conceptual

La identidad visual de la aplicación se fundamenta en una paleta de colores armoniosa y profesional. El azul claro (#f0f8ff) se utiliza como color de fondo predominante, evocando una sensación de apertura y amigabilidad, creando un ambiente acogedor para el usuario desde el primer contacto. La tipografía en negro (#000000) garantiza una legibilidad óptima del texto principal, asegurando que la información esencial sea fácilmente accesible y comprensible.

El azul vibrante (#1976D2) se reserva estratégicamente para los botones de acción principales, como "ACEPTAR" o "Agregar método de pago", capturando la atención del usuario y guiándolo hacia las interacciones clave. Este color primario transmite confianza y profesionalismo. Los mensajes de error se presentan en un rojo discreto pero visible, una señal universal de advertencia que informa al usuario sobre la necesidad de corrección sin interrumpir la fluidez de su interacción.

La elección de esta paleta de colores busca un equilibrio entre accesibilidad y profesionalismo, creando una interfaz que es tanto invitante como confiable. El uso estratégico del color azul para las acciones principales asegura que los usuarios puedan identificar rápidamente cómo interactuar con el sistema, mientras que la claridad del texto negro sobre el fondo azul claro garantiza una lectura cómoda y sin esfuerzo.

B. Estructura general

La navegación dentro de la aplicación se ha diseñado para ser intuitiva y consistente. Una barra superior presente en todas las pantallas muestra claramente el título de la sección actual y un botón de retroceso ubicado estratégicamente en la esquina superior izquierda, siguiendo un patrón de navegación familiar para los usuarios de aplicaciones móviles. Esta consistencia permite una orientación sencilla dentro de la aplicación, minimizando la curva de aprendizaje.

El contenido dentro de las pantallas se organiza de forma clara y lógica, a menudo utilizando listas scrollables para presentar información variable como el historial de movimientos o los métodos de pago guardados. La separación visual entre los elementos mediante líneas sutiles o fondos ligeramente diferenciados asegura que la información sea fácil de escanear y comprender.

La estructura de navegación lineal y la presentación organizada del contenido aseguran que los usuarios puedan encontrar la información que necesitan de manera rápida y eficiente. La consistencia en el diseño de la barra superior y el botón de retroceso promueve una experiencia de usuario predecible y sin frustraciones.

C. Experiencia UX/UI

Campos de Entrada Validados: La inclusión de texto de marcador de posición y la **validación en tiempo real con mensajes de error discretos** proporcionan una guía clara al usuario al ingresar información. Esta retroalimentación inmediata **reduce los errores y mejora la eficiencia** en la captura de datos.

- **Justificación para el Cliente:** Unos campos de entrada validados eficientemente **ahorran tiempo al usuario** y **minimizan la posibilidad de errores**, lo que se traduce en una experiencia más **fluida y precisa**.

Listas Scrollables: La presentación de información variable en listas optimizadas permite una **visualización organizada y un rendimiento fluido**, incluso con grandes cantidades de datos. La **separación visual clara** entre los elementos facilita la distinción y la lectura.

- **Justificación para el Cliente:** Un rendimiento fluido y una presentación clara de la información aseguran que los usuarios puedan **acceder y comprender sus datos de manera eficiente**, mejorando la **usabilidad** general de la aplicación.

Botones Interactivos: La **retroalimentación visual sutil** al tocar los botones principales (cambio de opacidad) confirma la interacción del usuario, proporcionando una sensación de **respuesta y control**. El **texto en negrita y de tamaño adecuado** asegura la legibilidad y la identificación clara de la función del botón.

- **Justificación para el Cliente:** Una retroalimentación clara en las interacciones aumenta la **confianza del usuario** en el sistema y asegura que sus acciones se registren correctamente, lo que contribuye a una experiencia más **satisfactoria**.

Logo de la Tarjeta en el Ítem de Método de Pago: La inclusión de los logos de Visa o MasterCard junto a los últimos cuatro dígitos de la tarjeta permite una **identificación visual rápida y familiar** de los métodos de pago guardados.

- **Justificación para el Cliente:** La identificación visual rápida de los métodos de pago **simplifica la selección** y **aumenta la confianza** del usuario al realizar transacciones.

Iconografía: El uso de iconos reconocibles para la navegación (flecha hacia atrás) y la indicación de acciones adicionales (flecha hacia adelante) mejora la **eficiencia de la navegación y la comprensión de la funcionalidad sin necesidad de texto extenso**.

- **Justificación para el Cliente:** El uso de iconos intuitivos **facilita la navegación** y la **comprensión de las funciones** de la aplicación, haciéndola más **accesible y eficiente** para el usuario.

D. Mockups

1) Aplicación de cliente

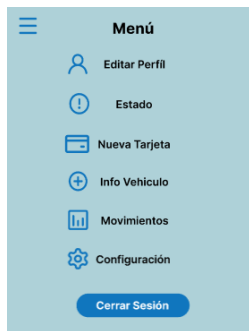
Inicio de sesión: La primera vista implementada que vera el usuario al iniciar la aplicación, teniendo los botones para realizar la navegación ya sea a Registro, Contraseña olvidada o entrar al menú principal luego de hacer el inicio de sesión.

Registro: Si un usuario no está registrado en la aplicación debe llenar los campos para poder iniciar sesión, teniendo la posibilidad de volver al Log in si ya tiene cuenta o crear una cuenta y devolverlo al Log in automáticamente.

Contraseña Olvidada: En caso de no recordar la contraseña para poder iniciar sesión se enviará un correo para la respectiva recuperación de la contraseña.

Menú principal (Inicio): Luego de verificar la cuenta y poder iniciar sesión se podrá ver la pantalla principal en donde se encontrarán los botones para ir a las demás vistas.

Sidebar: Si el usuario oprime el botón de 3 rayas se desplegará desde abajo un pequeño menú donde contendrá los mismos botones que harán la navegación a las mismas vistas, el único botón diferente será el de editar perfil y cerrar sesión para que el usuario edite perfil



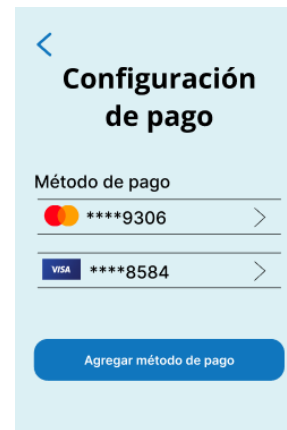
Consulta de vehículos: Se podrá observar los vehículos que el usuario tiene registrados, en caso de no tener algún vehículo registrado se podrá crear en la siguiente vista. Pero si se desea eliminar algún vehículo se debe seleccionar



Estado: En el estado se podrá ver la placa del vehículo extraída de la imagen que tomo la cámara a la hora de entrar al parqueadero, también se podrá ver si el vehículo esta estacionado o si no registra en el caso de que salga del parqueadero. También hay un botón para pagar y dejar de contar la tarifa, con la condición de que exista una tarjeta de crédito o debito para hacer el respectivo pago.



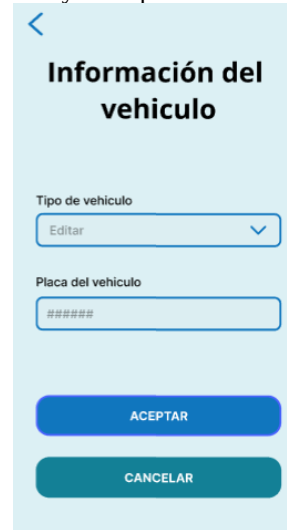
Configuración de pagos: En la configuración de pagos se podrá ver las tarjetas que se hayan agregado, en caso de no tener una tarjeta agregada se vera el mensaje de que no hay métodos de pago registrados. En ese caso estará el botón de agregar método de pago que pasará a la siguiente vista para agregar una tarjeta ya sea de crédito o débito.



Movimientos: En movimientos hay una tabla en donde se registran todos los pagos hechos con la fecha y el vehículo con el que se ingresó al parqueadero.



Agregar información vehículos: Para que se detecte el vehículo en la entrada del parqueadero se debe tener un vehículo registrado, para eso esta esta vista en donde se pide el tipo de vehículo, ya sea un carro o una moto y la placa de ese vehículo para que sea detectada y leída por la cámara.



Datos tarjeta: Si el usuario tiene agregada una tarjeta y desea ver la información esta será la vista de los datos de la tarjeta, teniendo la posibilidad de editar algunos datos de la tarjeta o eliminar ese método de pago.

<

Datos de tarjeta

Crédito/Debito

VISA

****8584

Titular de la tarjeta

Juan Valdés

Fecha de vencimiento

09/2027

EDITAR

ELIMINAR

Editar datos tarjeta: Si se desea editar algunos datos de la tarjeta, los campos disponibles para editar serán el titular, la fecha de vencimiento y el CVC o CVV dependiendo de la tarjeta.

<

Método de Pago

VISA

****8584

Titular de la tarjeta

Tu Nombre

Fecha de vencimiento

Mes

Año

CVV

— — —

SIGUIENTE

CANCELAR

Agregar tarjeta: Si el usuario no tiene algún método de pago registrado, esta vista será la encargada de tomar los datos de la tarjeta y guardarlos, al ingresar el numero de la tarjeta se verá si es una visa o MasterCard

<

Método de Pago

Número de la tarjeta

#####

MasterCard

VISA

Editar

Titular de la tarjeta

Tu Nombre

Fecha de vencimiento

Mes

Año

CVV

— — —

SIGUIENTE

CANCELAR

Edición Perfil: En la edición del perfil solo estarán disponibles algunos campos de texto como el nombre, apellido y la contraseña, los demás campos estarán bloqueados.

<

Perfil

Tipo de Documento

Editar

Numero de Documento

Introduce tu No de documento

Nombre

Tu Nombre

Apellido

Tus Apellidos

Correo

example@email.com

Contraseña

Introduce tu contraseña

ACEPTAR

2) Aplicación de administrador

Inicio de sesión: La primera vista implementada que vera el usuario al iniciar la aplicación, teniendo los botones para realizar la navegación a Contraseña olvidada o entrar al menú principal luego de hacer el inicio de sesión.

P

Bienvenido

Inicia sesión en ParkSide

Correo

example@email.com

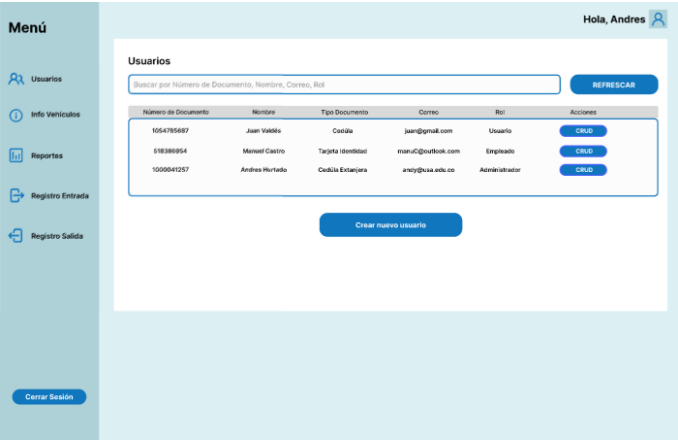
Contraseña

Introduce tu contraseña

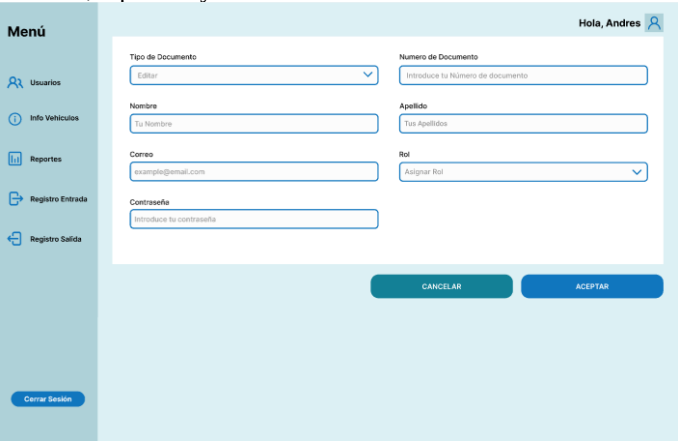
INICIAR SESIÓN

He olvidado mi contraseña

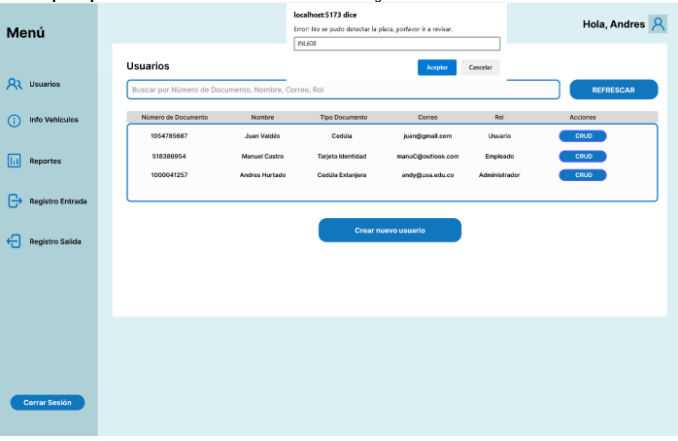
Usuarios: Luego del inicio de sesión la primera vista va a ser usuarios, donde se hay una tabla con el registro de todos los usuarios registrados en la aplicación, evidenciando algunos datos, siempre va a estar fijo la barra lateral para hacer la navegación a las demás vistas. Si se oprime el botón que dice CRUD, que en la actualidad es un + son para bloquear o desactivar al usuario seleccionado.



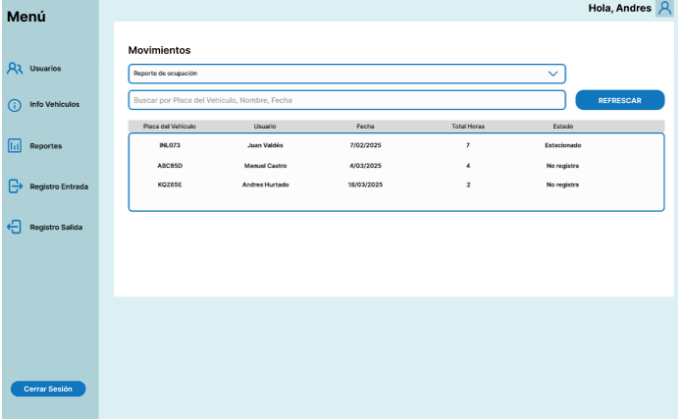
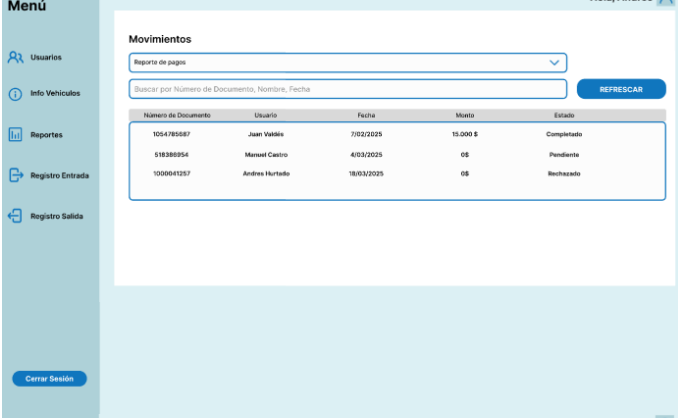
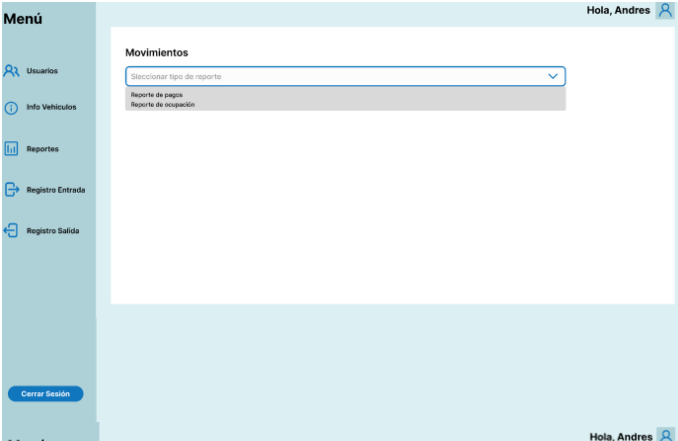
Editar Perfil (Solo para administrador): En la aplicación web solo se podrá editar los datos del administrador, pero también habrán campos bloqueados, los únicos campos editables son el Nombre, Apellido y Contraseña.



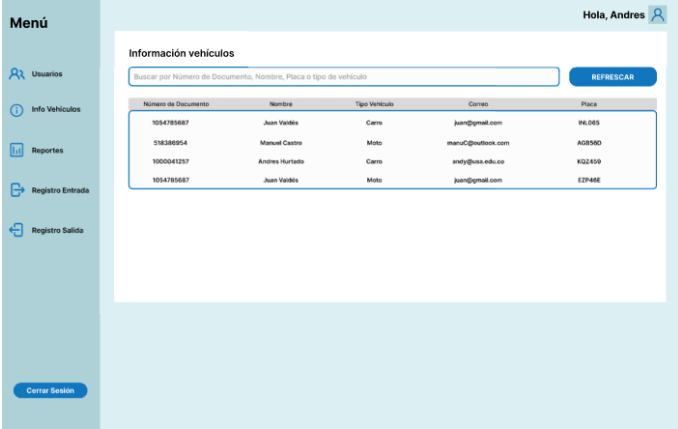
Error en la detección de la placa: Al intentar ingresar al parqueadero y no poder leer bien la placa del vehículo se enviara un mensaje al encargado describiendo el problema para que se acerque personalmente al vehículo y tome los datos.



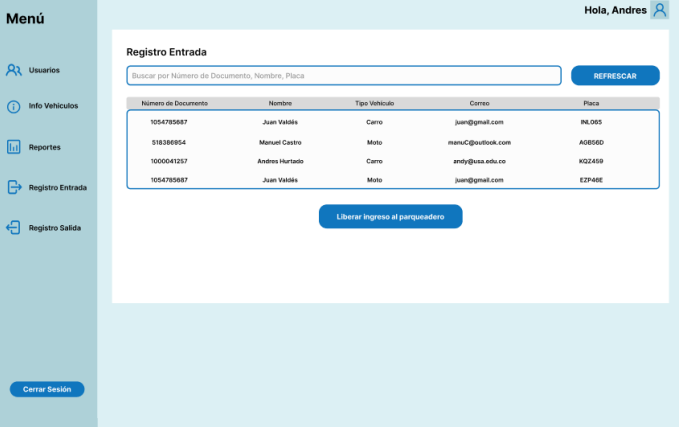
Reportes y movimientos: En los reportes y movimientos se tendrá que seleccionar alguna de las dos opciones las cuales son reportes de pagos y reportes de ocupación. Dependiendo de lo que se elija se renderizara una tabla con diferentes datos. Por ejemplo, en reportes de ocupación se podrá observar la placa del vehículo, el nombre del usuario, la fecha de parqueo, el total de horas en las que estuvo en el parqueadero y el estado.



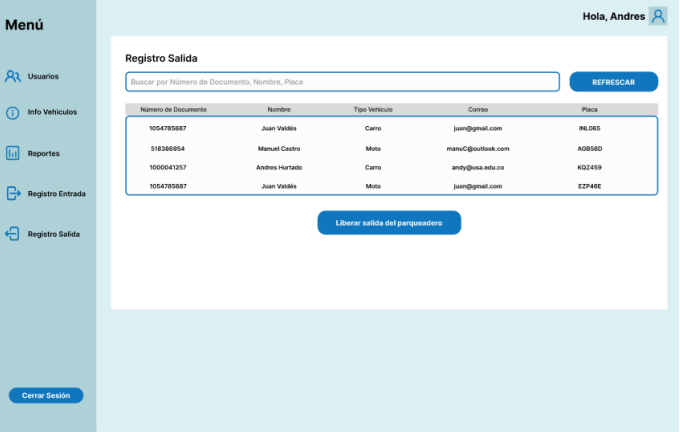
Información Vehículos: En esta pestaña se podrá ver la información de los vehículos registrados, con los datos de sus respectivos propietarios.



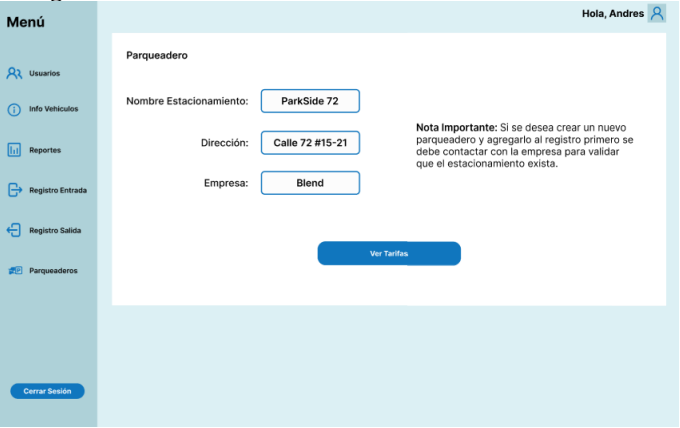
Registro Entrada: Como se menciono anteriormente, cuando haya problemas para leer la placa de un vehículo y se envíe el mensaje al encargado, se podrá liberar el ingreso al parqueadero haciendo la búsqueda de la placa o el usuario para seleccionarlo y permitir el ingreso.



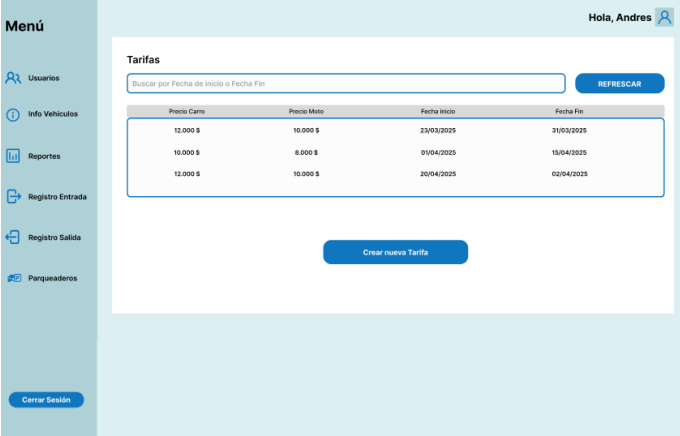
Registro Salida: Esta vista hará lo mismo que en registro entrada, pero en este caso será para la salida de un vehículo del parqueadero.



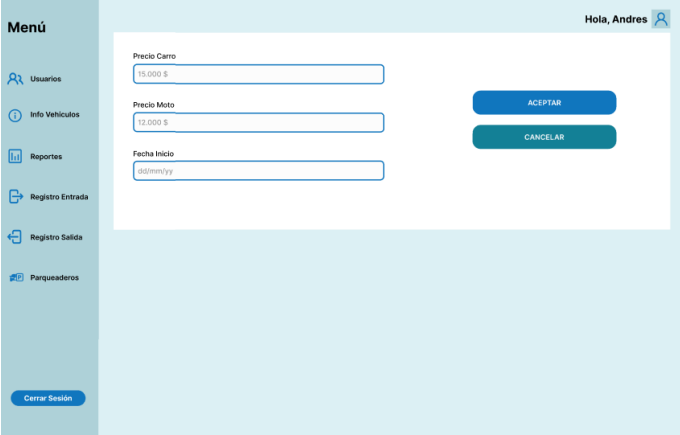
Parqueadero: Esta vista será para mostrar la información del parqueadero actualmente activo con los datos de la dirección y la empresa a la que pertenece, tiene un botón para hacer la navegación a las tarifas actualmente activas.



Tarifas: Se podrán observar las tarifas actualmente activas en donde se verán las tarifas para los carros, las motos, fecha de inicio y fecha fin, con la opción de crear nuevas tarifas.



Crear Tarifas: Para la creación de las tarifas se tendrá un formulario en donde se debe ingresar el precio del carro, el precio de la moto y la fecha de inicio de la tarifa.



Cada decisión visual y de diseño en la aplicación ParkSide AI se ha tomado con el firme objetivo de optimizar la experiencia del usuario, reflejando una imagen profesional, confiable y moderna de su servicio. La paleta de colores equilibrada, el estilo minimalista, la consistencia en los elementos de diseño y la retroalimentación visual clara se combinan para crear una interfaz que es a la vez atractiva y funcional. Creemos que este enfoque centrado en el usuario fomentará la adopción, la satisfacción y la lealtad hacia su producto.

VI. INTELIGENCIA ARTIFICIAL

El sistema Parkside AI utiliza dos modelos de visión por computadora con el objetivo de agilizar el flujo de trabajo dentro del parqueadero mediante la automatización del registro de entrada de vehículos, para esto se lleva a cabo el siguiente proceso.

Se carga la imagen tomada por una cámara en la entrada del parqueadero en un modelo de red neuronal convolucional profunda YOLOV8 que realiza la detección de la placa dentro de la imagen original arrojando las coordenadas de la placa en formato de bounding box, con estas coordenadas se realiza un recorte sobre la imagen original. El modelo fue entrenado con un dataset de más de 5 mil imágenes de placas colombianas

usando técnicas de augmentación y etiquetas en formato YOLO (coordenadas normalizadas). Presentando un rendimiento muy alto con métricas como Precision: 0.97, Recall: 0.95 mAP@0.5: 0.95 y mAP@0.5:0.95: 0.70 que nos indican una buena capacidad del modelo para detectar el área de la placa, las métricas mAP con diferentes umbrales IoU (Intersection over Union) nos muestran el rendimiento del modelo al comparar la superposición del bounding box predicho con bounding box original.

Una vez se tiene la imagen recortada con la placa se pasa por una función de preprocesamiento que la convierte a escala de grises, reduce el ruido de la imagen y ajusta su contraste de una manera adaptativa según su nivel de variación (desviación estándar de la imagen) y se envía a un modelo EasyOCR que realiza la extracción del texto y envía el texto de la placa al backend que continua con el flujo del parqueo.

Estos modelos fueron escogidos por distintas razones.

YOLO: Velocidad en tiempo real, alta precisión, detección robusta, facilidad de entrenamiento con datasets personalizados como los encontrados con placas colombianas.

EasyOCR: Soporte multilenguaje y buena precisión, integración sencilla mediante su API sin necesidad de entrenamiento adicional.

VII. MANEJO DE INFORMACIÓN

El sistema ParkSide AI gestiona su información a través de una única base de datos relacional, implementada en el motor PostgreSQL. Este modelo ha sido diseñado para garantizar la integridad, la eficiencia y la escalabilidad de los datos necesarios para el funcionamiento integral del sistema de gestión de estacionamientos automatizado.

A. Definición técnica

El modelo de datos de ParkSide AI puede abstraerse en las siguientes entidades principales y sus interrelaciones:

- **Entidades Centrales:**
 - **Cliente:** Representa a las personas que utilizan el sistema de estacionamiento.
 - **vehículo:** Representa los automóviles o motocicletas que acceden al estacionamiento.
 - **RegistroParqueo:** Registra los eventos de entrada y salida de los vehículos al estacionamiento.
 - **Pago:** Detalla las transacciones financieras realizadas por los clientes por el servicio de estacionamiento.

- **Entidades de Soporte:**

- **Tarjeta:** Almacena la información de los métodos de pago asociados a los clientes.
- **Parqueadero:** Contiene la información de las instalaciones del estacionamiento.
- **HistoricoTarifas:** Lleva un registro de los cambios en las tarifas del estacionamiento a lo largo del tiempo.
- **Empleado:** Almacena la información de los administradores y personal del estacionamiento.

Las interrelaciones clave se centran en cómo los clientes registran sus Vehículos y asocian Tarjetas para realizar Pagos relacionados con sus RegistroParqueos en un Parqueadero específico, cuyas tarifas se gestionan a través del HistoricoTarifas y es administrado por Empleados. El RegistroParqueo actúa como un punto de conexión entre los Vehículos y los Pagos.

B. Arquitectura de datos

La arquitectura de datos de ParkSide AI se caracteriza por un diseño relacional normalizado, optimizado para la consistencia y la minimización de la redundancia de la información. El esquema se organiza en torno a las transacciones de estacionamiento, donde el registro de entrada y salida (RegistroParqueo) sirve como eje para vincular la información del vehículo, el cliente y el pago.

La gestión de la información de los clientes y sus vehículos se mantiene separada, permitiendo que un mismo cliente pueda tener múltiples vehículos registrados y asociar diferentes métodos de pago. La información específica del estacionamiento, como la capacidad y las tarifas, se almacena en tablas dedicadas, facilitando la gestión de múltiples instalaciones en el futuro. El histórico de tarifas permite un seguimiento de los precios a lo largo del tiempo, crucial para la gestión operativa y la facturación. La tabla de empleados proporciona un mecanismo para la gestión de acceso y roles dentro del sistema administrativo.

La integridad de los datos se mantiene mediante la aplicación de claves primarias y foráneas, asegurando que las relaciones entre las entidades sean consistentes y que no existan registros huérfanos. El uso de tipos de datos específicos para cada atributo optimiza el almacenamiento y facilita la realización de consultas eficientes. La arquitectura de datos está diseñada para soportar un volumen significativo de transacciones y clientes, con la capacidad de escalabilidad inherente a PostgreSQL.

C. Diagrama ERD

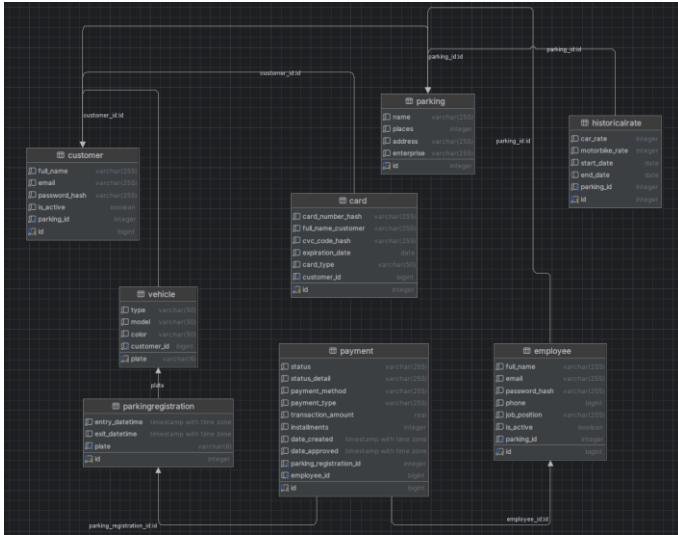


Figura 1: Diagrama EDR

VIII. MEJORAS FUTURAS

A. Frontend

1) Aplicación de administrador

A futuro, la plataforma podrá incluir funcionalidades más robustas para la gestión de usuarios, tales como la edición y eliminación directa desde la interfaz del administrador. Esto permitirá una administración más flexible y dinámica del sistema, con la posibilidad de modificar roles, datos personales y estado de los usuarios sin requerir intervención técnica. Asimismo, se proyecta la integración de filtros avanzados en las tablas, permitiendo búsquedas más precisas por nombre, documento, fecha de registro, entre otros criterios relevantes para cada módulo.

Adicionalmente, se contempla la incorporación de un sistema de notificaciones internas para informar a los usuarios o al administrador sobre acciones críticas, como intentos de acceso fallidos, vencimientos de tarifas o bloqueos recientes. También se evaluará la implementación de roles personalizados con diferentes niveles de permisos, para ofrecer una experiencia más controlada y segura. Todas estas mejoras están orientadas a fortalecer la usabilidad, el control administrativo y la escalabilidad del sistema.

2) Aplicación de usuario

En cuanto a la aplicación móvil, se prevé el desarrollo de nuevas funcionalidades que optimicen la experiencia del usuario final, como la visualización en tiempo real del estado del parqueadero, incluyendo disponibilidad por zonas y tipos de vehículo. Asimismo, se contempla la posibilidad de integrar un sistema de pagos móviles a través de pasarelas seguras, permitiendo a los usuarios realizar recargas o pagos desde la misma aplicación de forma rápida y confiable.

B. Backend

Además de las mejoras en la inteligencia artificial descritas posteriormente, el backend del sistema ParkSide AI contempla diversas extensiones funcionales y técnicas que serán abordadas en etapas futuras. Una de las prioridades es la implementación completa del módulo de pagos, el cual se pretende integrar con plataformas como **Mercado Pago**, permitiendo la automatización de transacciones y el registro centralizado de los pagos realizados por los usuarios. No obstante, su implementación depende de una migración del sistema a un entorno **HTTPS**, condición indispensable exigida por esta pasarela de pagos para garantizar la seguridad de las transacciones.

Asimismo, se prevé el desarrollo de nuevas rutas para ampliar la funcionalidad actual del backend. Estas incluyen endpoints para el manejo avanzado de registros de vehículos por parte de los usuarios (permitiendo futuras actualizaciones o eliminaciones), así como mejoras en el módulo de gestión del parqueadero que permitirá registrar múltiples sedes con información diferenciada. El módulo de historial de tarifas también podrá evolucionar para incluir auditorías automáticas y ajustes dinámicos de precios según la hora o demanda.

El flujo interno del backend se estructura en **tres componentes principales**:

1. **API**: Encargada de exponer los endpoints que comunican el frontend y los modelos de IA con la lógica de negocio.
2. **Modelos**: Definen la estructura de los datos y actúan como intermediarios entre la lógica y la base de datos, asegurando la consistencia de la información.
3. **Cruds**: Encapsulan las consultas y operaciones sobre la base de datos, facilitando la reutilización, el mantenimiento y la eficiencia de las operaciones.

Estas mejoras futuras están orientadas a fortalecer la robustez, escalabilidad y seguridad del sistema, permitiendo una operación más eficiente y adaptable a nuevos escenarios de uso.

C. IA

1) Implementación de Nuevos Modelos de Extracción de Texto

Se probarán nuevos modelos para extraer el texto buscando una mejora en el rendimiento ya que se detectaron errores puntuales como con la letra w, no se identifica correctamente en casos de poca exposición de luz en la imagen original.

2) Reducción de Tiempos de Inferencia

Actualmente los modelos demoran aproximadamente 3 segundos en realizar todo el proceso de inferencia y envío de la placa al backend de la aplicación, a futuro se buscará reducir este tiempo usando modelos de menor complejidad que combinen una velocidad adecuada con alta precisión.

IX. ANEXOS

Repositorio del proyecto:

<https://github.com/mscp-ccia/parksideproyect.git>

Figma del diseño:

<https://www.figma.com/design/eR3xDIjBiBA6o6h5l1tGqe/ParkSide?node-id=0-1&t=q0iTGtbq9ZhhBBzB-1>